Tema IV. JDBC

Desarrollo de Aplicaciones para Internet
Curso 26|27

Miguel Reboiro Jato

BY NC SA

Universidade Vigo
ESEI | ENXEÑERÍA INFORMÁTICA
ESCOLA SUPERIOR OURENSE

Índice

1. Introducción
2. Arquitectura
3. Consultas SQL
    i. Carga de Drivers
    ii. Conexión
    iii. Consulta
    iv. Resultados
4. Tipos de Datos
5. Transacciones
6. Referencias

Índice

1. **Introducción**
2. Arquitectura
3. Consultas SQL
    i. Carga de Drivers
    ii. Conexión
    iii. Consulta
    iv. Resultados
4. Tipos de Datos
5. Transacciones
6. Referencias

Introducción

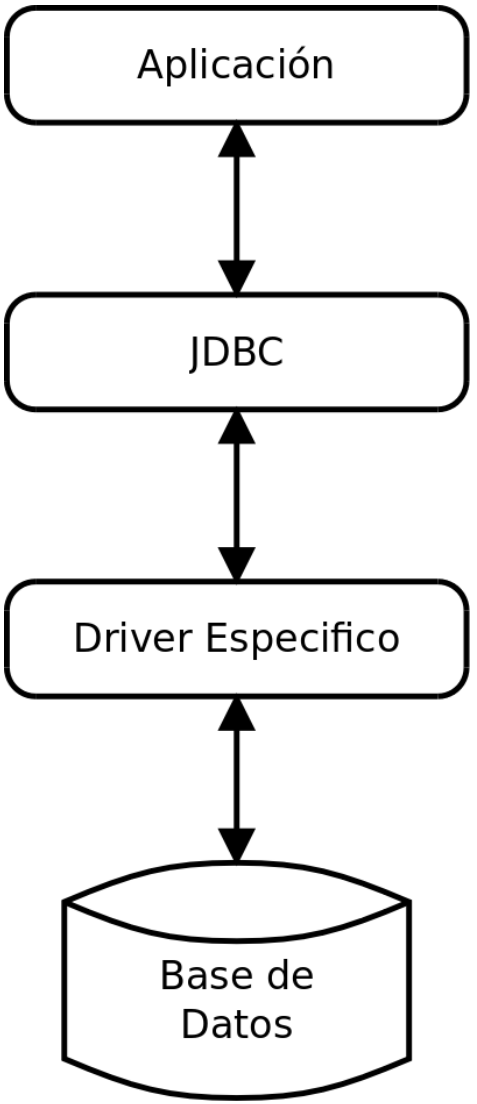
- **Java Database Connectivity (JDBC)** es la API de Java para trabajar con bases de datos relacionales.
- Es una API **genérica**.
- Esta API permite:
  - **Conectarse** a una fuente de datos (normalmente una base de datos).
  - **Enviar consultas y sentencias de actualización** a la base de datos.
  - **Recuperar y procesar los resultados** recibidos de la base de datos en respuesta a las consultas.
  - **Obtener información de los elementos** de la base de datos (tablas, columnas, etc.).

Índice

1. Introducción
2. **Arquitectura**
3. Consultas SQL
    i. Carga de Drivers
    ii. Conexión
    iii. Consulta
    iv. Resultados
4. Tipos de Datos
5. Transacciones
6. Referencias

Arquitectura

- JDBC introduce una **capa de abstracción** sobre el SGBD.
- **Evita** crear una **dependencia** con un SGBD concreto.
- Cada proveedor de SGBD debe proporcionar un ***driver* específico**.
- La API de JDBC está formada por una serie de **interfaces** que deberá implementar cada *driver* JDBC.

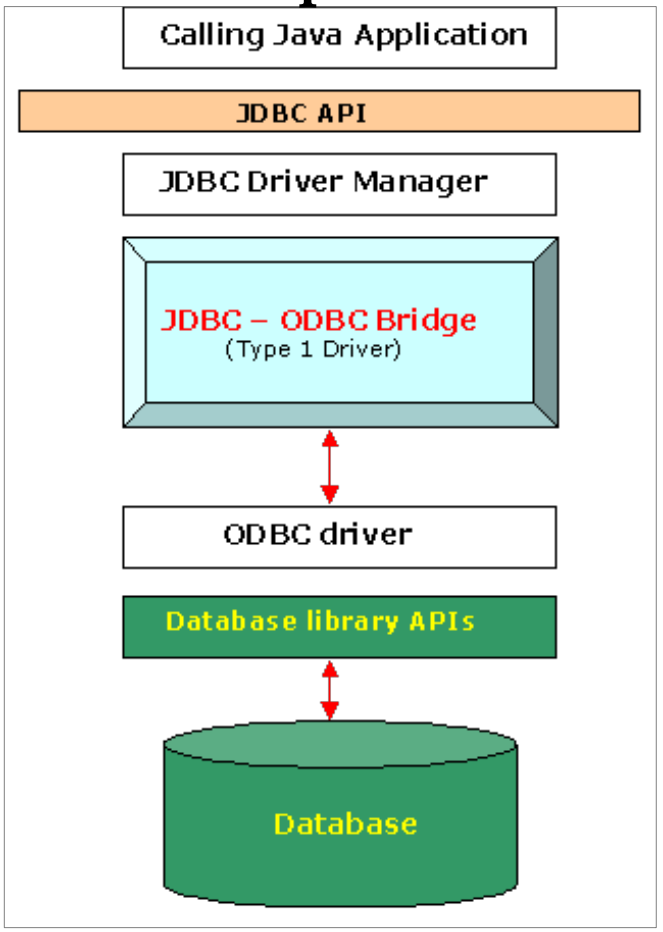


Drivers

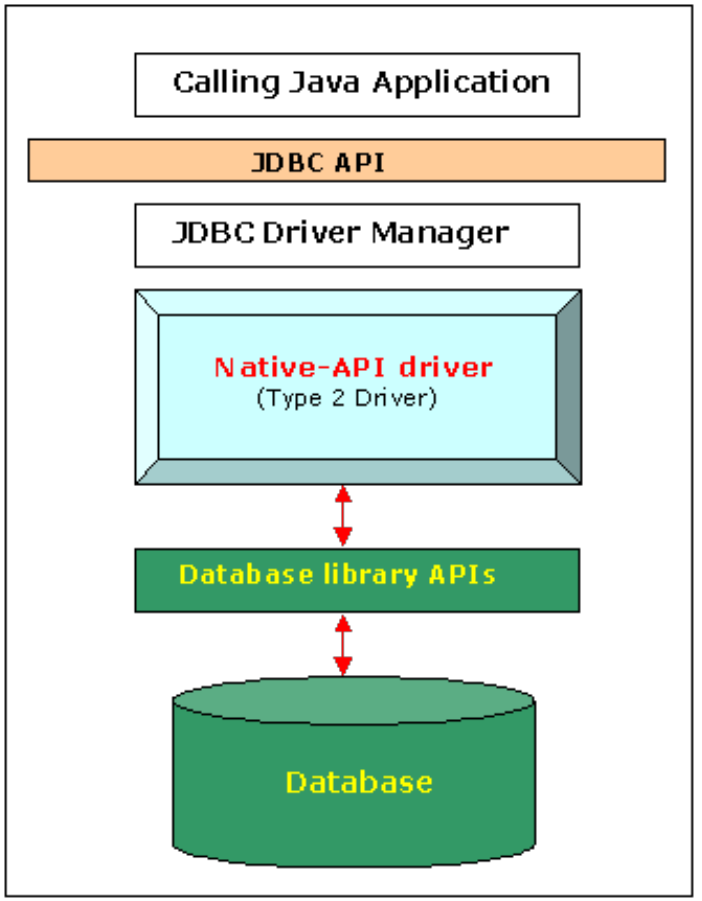
- Existen **cuatro tipos** de *drivers* JDBC:
  - Tipo 1
    - *Drivers* que implementan la API JDBC como un mapeo a otra API de acceso a datos genérica (p.ej. ODBC).
    - No se recomienda salvo que sea la única solución.
  - Tipo 2
    - *Drivers* escritos parcialmente en Java y parcialmente en código nativo.
    - No se recomiendan por ser poco portables.
  - Tipo 3
    - *Drivers* que implementan un cliente en Java que se comunican con la base de datos utilizando un protocolo independiente de la base de datos.
  - Tipo 4
    - *Drivers* que implementan un cliente en Java que utiliza un protocolo específico para la base de datos.

Drivers

Tipo 1



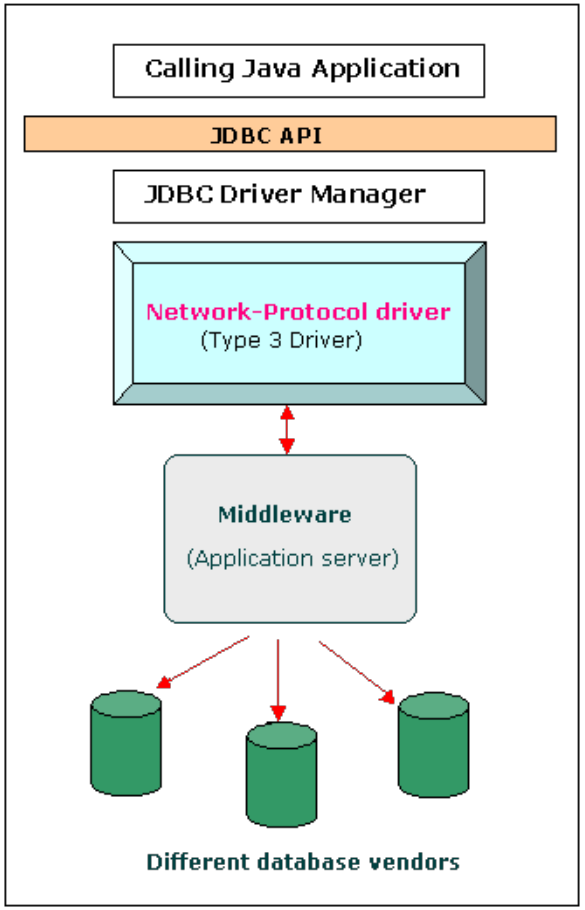
Tipo 2



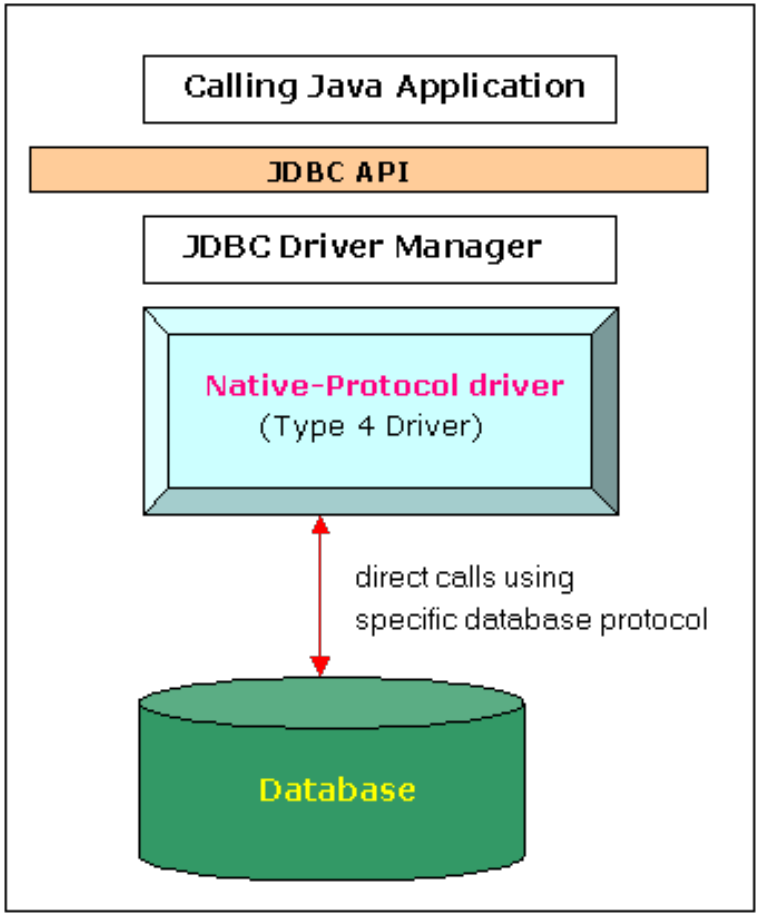
*Imágenes realizadas por Jay para la versión inglesa de Wikipedia

Drivers

Tipo 3



Tipo 4



*Imágenes realizadas por Jay para la versión inglesa de Wikipedia

Índice

1. Introducción
2. Arquitectura
3. **Consultas SQL**
    i. Carga de Drivers
    ii. Conexión
    iii. Consulta
    iv. Resultados
4. Tipos de Datos
5. Transacciones
6. Referencias

Consultas SQL

- En general, el **procesado de una sentencia SQL** sigue los siguientes pasos:
  1. Establecer una conexión.
  2. Crear una sentencia.
  3. Ejecutar la consulta (sentencia).
  4. Procesar los resultados (`ResultSet` o `int`).
  5. Cerrar la conexión.

- **Antes de poder realizar una consulta es necesario cargar el *driver* del SGBD**.

Índice

1. Introducción
2. Arquitectura
3. **Consultas SQL**
    i. **Carga de Drivers**
    ii. Conexión
    iii. Consulta
    iv. Resultados
4. Tipos de Datos
5. Transacciones
6. Referencias

Cargar Drivers

- El **JRE no incluye ningún *driver*** para base de datos.
- Cada SGBD debe proporcionar el suyo.
- Los ***drivers* más comunes** son:
  - **JavaDB**
    - Incluido en el JDK hasta la versión 8.
    - Es un SGBD embebido (concretamente Apache Derby).
  - **MySQL**
    - https://dev.mysql.com/downloads/connector/j/
  - **PostgreSQL**
    - https://jdbc.postgresql.org/
  - **Oracle**
    - https://www.oracle.com/es/database/technologies/appdev/jdbc-downloads.html

Cargar Drivers

- Cada *driver* proporciona un **fichero jar** con las clases que implementan el API JDBC.
- Para que un *driver* esté disponible debe **registrarse en el `DriverManager`**.
- Desde JDBC 4.0 (Java SE 6), **el registro se realiza automáticamente**.
- En **versiones antiguas**, el registro se forzaba cargando la clase del *driver* (que implementa `java.sql.Driver`). Por ejemplo:

```
Class.forName(    // JavaDB
  "org.apache.derby.jdbc.EmbeddedDriver")
Class.forName(    // MySQL
  "com.mysql.cj.jdbc.Driver")
```

Índice

1. Introducción
2. Arquitectura
3. **Consultas SQL**
    i. Carga de Drivers
    ii. Conexión
    iii. Consulta
    iv. Resultados
4. Tipos de Datos
5. Transacciones
6. Referencias

Establecer Conexión

- La conexión con un SGBD en JDBC puede realizarse a través de **dos clases**.
  - **`DriverManager`**
    - Establece una conexión con la base de datos a partir de una URL (método `getConnection`).
    - Los *drivers* deben haberse registrado previamente.
  - **`DataSource`**
    - Los parámetros de la conexión con la base de datos se definen fuera de la aplicación.
    - Es la opción recomendada, pues es transparente para la aplicación.
    - Son necesarias bibliotecas no estándar (p.ej. Apache Commons DBCP, c3p0, HikariCP, etc.).

Cadenas de Conexión

- Cada *driver* utiliza su propio formato de cadena de conexión (URL).
- Algunos ejemplos:
  - **JavaDB (embebido)**
    - `jdbc:derby:[subsubprotocol:][databaseName]`
      `[;attribute=value]*`
    - Ejemplo:
      - `jdbc:derby:personas;create=true`
  - **MySQL**
    - `jdbc:mysql://[host][,failoverhost...][:port]/`
      `[database][?propertyName1][=propertyValue1]`
      `[&propertyName1][=propertyValue2]...`
    - Ejemplo:
      - `jdbc:mysql://localhost:3306/personas`

DriverManager

(Ejemplos)

```
// Conexión de ejemplo para JavaDB
DriverManager.getConnection(
   "jdbc:derby:example;create=true",
   "user", "password"
);

// Conexión de ejemplo para MySQL
DriverManager.getConnection(
   "jdbc:mysql://localhost/example",
   "user", "password"
);
```

Connection

- Los objetos de este tipo **representan una conexión** con una base de datos.
- Debe **cerrarse** una vez finalice su uso (try con recursos o método `close`).
- Contiene los métodos más importantes:
  - `createStatement/prepareCall/ prepareStatement`
  - `commit/rollback`
  - `get/setAutoCommit`
  - `get/setTransactionIsolation`
  - `getMetaData`

Índice

1. Introducción
2. Arquitectura
3. **Consultas SQL**
    i. Carga de Drivers
    ii. Conexión
    iii. Consulta
    iv. Resultados
4. Tipos de Datos
5. Transacciones
6. Referencias

Sentencias

- Las sentencias se crean a través del API de `Connection`.
- Existen **tres tipos de sentencias**:
  - **`Statement`**
    - Representa una sentencia SQL estática.
  - **`PreparedStatement`**
    - Representa una sentencia SQL.
    - Está precompilada y puede recibir parámetros.
  - **`CallableStatement`**
    - Permite invocar procedimientos almacenados.

Statement

- Las instancias de esta clase permiten **ejecutar consultas SQL**.
- Los métodos principales son:
  - **`executeQuery`**
    - Ejecuta consultas SQL que devuelvan una lista de resultados (SELECT).
    - Devuelve un objeto `ResultSet`.
  - **`executeUpdate`**
    - Ejecuta consultas SQL de tipo INSERT, UPDATE o DELETE, o sentencias DDL.
    - Devuelve un valor entero con el número de filas afectadas.

Statement

- Los **métodos principales** son: (continuación)
  - **`execute`**
    - Este método se emplea con consultas que no puedan ejecutarse con alguno de los otros dos métodos.
    - Permite ejecutar consultas que devuelvan múltiples resultados.
      - Devuelve `true` si el primer resultado es un `ResultSet` y `false` si es un contador de actualizaciones.
      - `getResultSet`: Devuelve el resultado si es `ResultSet`.
      - `getUpdateCount`: Devuelve el resultado si es un contador.
      - `getMoreResults`: Avanza al siguiente resultado.

Ejemplo DDL

```
private static void statementCreate() throws SQLException {
    // 1. Conexión a la base de datos
    try (Connection connection = DriverManager.getConnection(
        DB_URL, DB_USER, DB_PASSWORD
    )) {
        // 2. Creación de la consulta
        try (Statement statement = connection.createStatement()) {
            // 3. Creación de la tabla Ejemplo
            int result = statement.executeUpdate(
                "CREATE TABLE Ejemplo(" +
                    "id INT NOT NULL AUTO_INCREMENT," +
                    "nombre VARCHAR(255) NOT NULL," +
                    "PRIMARY KEY (id)" +
                 ")");
            // 4. Comprobación de resultado
            if (result != 0)
                throw new SQLException("Unexpected result value: " + result);
        }
    }
}
```

Ejemplo DCL

```
private static void statementGrant() throws SQLException {
    // 1. Conexión a la base de datos
    try (Connection connection = DriverManager.getConnection(
        DB_URL, DB_USER, DB_PASSWORD
    )) {
        // 2. Creación de la consulta
        try (Statement statement = connection.createStatement()) {
            // 3. Modificación de permisos para dai2
            int result = statement.executeUpdate(
                "GRANT SELECT ON dai.Ejemplo TO 'dai2'@'localhost' " +
                "IDENTIFIED BY 'dai2'"
            );

            // 4. Comprobación de resultado
            if (result != 0)
                throw new SQLException("Unexpected result value: " + result);
        }
    }
}
```

Ejemplo DML

```
private static void statementInsert() throws SQLException {
   // 1. Conexión a la base de datos
   try (Connection connection = DriverManager.getConnection(
      DB_URL, DB_USER, DB_PASSWORD
   )) {
      // 2. Creación de la consulta
      try (Statement statement = connection.createStatement()) {
         // 3. Creación de la tabla Ejemplo. Con RETURN_GENERATED_KEYS
         // hacemos que las claves primarias se puedan recuperar
         int result = statement.executeUpdate(
            "INSERT INTO Ejemplo (id, nombre) " +
               "VALUES (0, 'Ana'), (0, 'Juan')",
               Statement.RETURN_GENERATED_KEYS
         );

         // 4. Comprobación de resultado
         if (result == 2) {
            try (ResultSet generatedKeys = statement.getGeneratedKeys()) {
               while (generatedKeys.next()) {
                  System.out.println("Clave " + generatedKeys.getInt(1));
               }
            }
         } else {
            throw new SQLException("Unexpected result value: " + result);
         }
      }
   }
}
```

Ejemplo DQL

```
private static void statementSelect() throws SQLException {
    // 1. Conexión a la base de datos local "dai"
    // con login y password "dai"
    try (Connection connection = DriverManager.getConnection(
        DB_URL, DB_USER, DB_PASSWORD
    )) {
        // 2. Creación de la consulta
        try (Statement statement = connection.createStatement()) {
            // 3. Ejecución de la consulta
            try (ResultSet result = statement.executeQuery(
                "SELECT * FROM Ejemplo WHERE nombre LIKE 'A%'"
            )) {
                // 4. Visualización de los resultados
                while (result.next()) {
                    System.out.printf("Id: %d, Nombre: %s%n",
                        result.getInt(1), result.getString(2)
                    );
                }
            }
        }
    }
}
```

PreparedStatement

- Es una subclase de `Statement` que le añade la capacidad de **preprocesar sentencias**.
- El método `Connection#prepareStatement` recibe una **consulta SQL con marcadores de posición**.
- Los **marcadores de posición** se indican con **`?`**.
- El **índice inicial** de los marcadores de posición es **1**.
- Los marcadores de posición reciben un valor a través de los métodos **métodos "`set`"**.
- La consulta se ejecuta con `execute/executeQuery/executeUpdate` que, en este caso, **no reciben parámetros**.

Ejemplo DML

```
private static void preparedStatementInsert() throws SQLException {
   // 1. Conexión a la base de datos
   try (Connection connection = DriverManager.getConnection(
      DB_URL, DB_USER, DB_PASSWORD
   )) {
      // 2. Creación de la consulta. Con RETURN_GENERATED_KEYS
      // hacemos que las claves primarias se puedan recuperar
      try (PreparedStatement statement = connection.prepareStatement(
            "INSERT INTO Ejemplo (id, nombre) " +
               "VALUES (0, ?), (0, ?)",
            Statement.RETURN_GENERATED_KEYS)) {
         // 3. Asignación de los valores
         statement.setString(1, "Ana");
         statement.setString(2, "Juan");

         // 4. Creación de la tabla Ejemplo.
         int result = statement.executeUpdate();

         // 5. Comprobación de resultado
         if (result == 2) {
            try (ResultSet generatedKeys = statement.getGeneratedKeys()) {
               while (generatedKeys.next()) {
                  System.out.println("Clave " + generatedKeys.getInt(1));
               }
            }
         } else {
            throw new SQLException("Unexpected result value: " + result);
         }
      }
   }
}
```

Ejemplo DQL

```
private static void preparedStatementSelect() throws SQLException {
    // 1. Conexión a la base de datos
    try (Connection connection = DriverManager.getConnection(
        DB_URL, DB_USER, DB_PASSWORD
    )) {

        // 2. Preparación de la consulta
        try (PreparedStatement statement = connection.prepareStatement(
            "SELECT * FROM Ejemplo " +
            "WHERE id > ? AND nombre LIKE ?"
        )) {
            // 3. Asignación de los valores
            statement.setInt(1, 5);
            statement.setString(2, "A%");

            // 4. Ejecución de la consulta
            try (ResultSet result = statement.executeQuery()) {
                // 5. Visualización de resultado
                while (result.next()) {
                    System.out.printf("Id: %d, Nombre: %s%n",
                        result.getInt("id"),
                        result.getString("nombre")
                    );
                }
            }
        }
    }
}
```

Índice

1. Introducción
2. Arquitectura
3. **Consultas SQL**
    i. Carga de Drivers
    ii. Conexión
    iii. Consulta
    iv. Resultados
4. Tipos de Datos
5. Transacciones
6. Referencias

ResultSet

- Las instancias **`ResultSet`** contienen el **resultado de las consultas DQL**.
- Actúa como un iterador para **recorrer las filas resultantes** de una consulta.
- También permite **actualizar campos** de las filas resultantes, **insertar nuevas filas** y **eliminar filas**.
- Cuando finalice su uso, **debe ser cerrado** (método `close`) para liberar recursos.

ResultSet

- **Métodos principales:**
  - **`first(), last(), next(), previous()`**
    - Permiten manipular la posición del cursor que apunta a la fila actual.
    - El cursor comienza antes de la primera posición y finaliza después de la última.
  - **`isFirst(), isLast(), isBeforeFirst(), isAfterLast()`**
    - Permiten comprobar la posición del cursor.
  - **`absolute(int), relative(int)`**
    - Permiten mover el cursor un número de posiciones.
    - Admiten valores positivos y negativos.

ResultSet

- **Métodos principales: (continuación)**
  - **`get<Tipo>(int | String)`**
    - Devuelve el valor de una columna de la fila actual.
    - La columna puede indicarse por nombre o índice.
  - **`findColumn(String)`**
    - Devuelve el índice de la columna indicada.
  - **`wasNull()`**
    - Indica si el último valor obtenido de un método `get<Tipo>` fue `null`.
    - Es necesario para saber si los métodos que devuelven tipos primitivos (p.ej. `getInt`) han recuperado un valor `null` de la base de datos.

ResultSet

- **Métodos principales: (continuación)**
  - **`getResultMetaData()`**
    - Devuelve un objeto `ResultSetMetaData`.
    - Este objeto permite obtener meta-información del resultado (nombre de columnas, tipo de columnas, nº de columnas, etc.).
  - **`close()`**
    - Cierra el resultado y libera recursos.
    - Si se cierra la conexión asociada también se cerrará.

Índice

1. Introducción
2. Arquitectura
3. Consultas SQL
    i. Carga de Drivers
    ii. Conexión
    iii. Consulta
    iv. Resultados
4. **Tipos de Datos**
5. Transacciones
6. Referencias

Tipos de Dato

- JDBC define cómo se debe hacer el **mapeo de tipos** entre Java y SQL.
  - http://docs.oracle.com/javase/6/docs/technotes/guides/jdbc/getstart/mapping.html#996857
- Esta conversión debe tenerse en cuenta cuando:
  - Se **asignan valores** (p.ej. `PreparedStatement#set<Tipo>`).
  - Se **recuperan** (p.ej. `ResultSet#get<Tipo>`).
- La clase `java.sql.Types` contiene una serie de **constantes** que representan los **tipos de datos SQL**.
  - Se utilizan, principalmente, cuando se trabaja con la meta-información de las columnas.

Trabajo con Fechas (I)

- Java tiene dos APIs de fechas:
  - `java.util`
    - Incluye, entre otras cosas, clases para trabajar con fechas.
    - Fue el primer API para trabajo con fechas.
    - Soporte básico para trabajo con fechas.
    - Clases principales: `Date` y `Calendar`
  - `java.time`
    - Añadida en Java 8.
    - Soporte avanzado para trabajo con fechas.
    - Clases principales: `LocalDate`, `LocalTime` y `LocalDateTime`

Trabajo con Fechas (II)

- En JDBC se trabaja de forma distinta con cada API:
  - `java.util`
    - `ResultsSet` y `PreparedStatement` tienen métodos específicos para este API: `get/setDate`, `get/setTime` y `get/setTimestamp`.
    - Devuelven/Reciben subclases de Date.
  - `java.time`
    - El trabajo se realiza a través de los métodos: `getObject(columna, clase)` y `setObject(índice, objeto)`.
    - clase y objecto son un `LocalDate`, `LocalTime` y `LocalDateTime`.

Tipos de Dato SQL

Clases	Tipo de dato al que se aplica
BIGINT	Entero de 64 bits. Valores entre –9233720368547758O8 y 9233720368547758O7.
BINARY	Valor binario de 254 bytes.
BIT	Un bit: valor cero o uno.
CHAR	Cadena de caracteres de longitud fija 254 caracteres.
DATE	Fecha, consistente en día, mes y año.
DECIMAL	Número decimal.
DOUBLE	Decimal de coma flotante con un máximo de 15 dígitos de mantisa.
FLOAT	Decimal de coma flotante con un máximo de 15 dígitos de mantisa.
INTEGER	Entero de 32 bits. Valores entre –2147483648 y 2147483647.
LONGVARBINARY	Cadena binaria de más de 254 bytes.
LONGVARCHAR	Cadena de caracteres de más de 254 caracteres.
NUMERIC	Número decimal.
REAL	Decimal de coma flotante con un máximo de 7 dígitos de mantisa.
SAMLLINT	Entero de 16 bits. Valores entre –32768 y 32767.
TIME	Hora, consistente en horas, minutos y segundos.
TIMESTAMP	Fecha y hora con nanosegundos. Consiste en DATE más TIME, más un campo para los nanosegundos.
TINYINT	Entero positivo de 8 bits. Valores entre 0 y 255.
VARBINARY	Cadena binaria de longitud variable.

*Tabla tomada del libro “Programación Avanzada con Java 1.4.x y JavaTraceIt!”

ResultSet Getters

Métodos	Tipo de dato
PUBLIC INPUTSTREAM **GETASCIISTREAM**()	LONGVARCHAR
PUBLIC BIGDECIMAL **GETBIGDECIMAL**()	NUMERIC y DECIMAL
PUBLIC INPUTSTREAM **GETBINARYSTREAM**()	LONGVARBINARY
PUBLIC BOOLEAN **GETBOOLEAN**()	BIT
PUBLIC BYTE **GETBYTE**()	TINYINT
PUBLIC BYTE[] **GETBYTES**()	todos los BINARY
PUBLIC PUBLIC DATE **GETDATE**()	DATE
PUBLIC DOUBLE **GETDOUBLE**()	FLOAT y DOUBLE
PUBLIC FLOAT **GETFLOAT**()	REAL
PUBLIC INT **GETINT**()	BIGINT
PUBLIC LONG **GETLONG**()	REAL
PUBLIC OBJECT **GETOBJECT**()	todos los tipos
PUBLIC SHORT **GETSHORT**()	SMALLINT
PUBLIC STRING **GETSTRING**()	CHAR y VARCHAR
PUBLIC PUBLIC TIME **GETTIME**()	TIME
PUBLIC TIMESTAMP **GETTIMESTAMP**()	TIMESTAMP
PUBLIC INPUTSTREAM **GETUNICODESTREAM**()	LONGVARCHAR

*Tabla tomada del libro “Programación Avanzada con Java 1.4.x y JavaTraceIt!”

Índice

1. Introducción
2. Arquitectura
3. Consultas SQL
    i. Carga de Drivers
    ii. Conexión
    iii. Consulta
    iv. Resultados
4. Tipos de Datos
5. **Transacciones**
6. Referencias

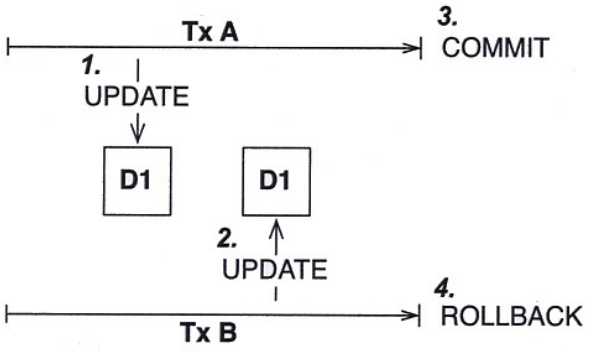
Transacciones

- El trabajo con las transacciones se hace mediante los métodos de `Connection`.
  - **`setAutocommit(boolean autoCommit)`**
    - Permite indicar si se debe hacer un *commit* automático después de cada consulta.
    - Si se desea utilizar **transacciones**, deberá **desactivarse** (`setAutocommit(false)`).
  - **`commit()`**
    - Realiza un *commit*: se ejecutan todas las consultas realizadas desde el último *commit* o *rollback* o desde que se inició la conexión.
  - **`rollback()`**
    - Realiza un *rollback*: se cancelan todas las consultas realizadas desde el último *commit* o *rollback* o desde que se inició la conexión.
  - **`setTransactionIsolation(int level)`**
    - Establece el nivel de aislamiento deseado para las transacciones.

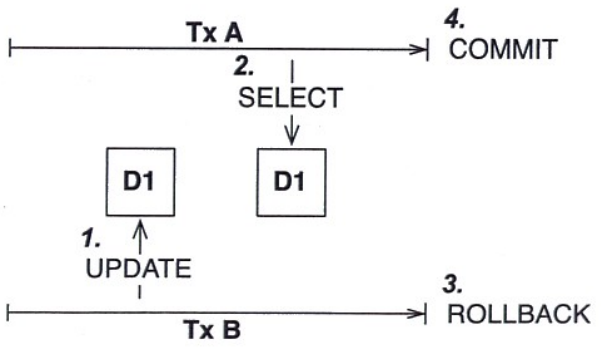
Transacciones

- **Problemas**
  - **Actualización Perdida (*Lost Update*)**
    - Dos transacciones actualizan una fila.
    - La segunda se aborta, de modo que el efecto de las dos actualizaciones se anula.
  - **Lectura Sucia (*Dirty Read*)**
    - Una transacción lee datos escritos por otra sin que se hayan confirmado.

Actualización Perdida



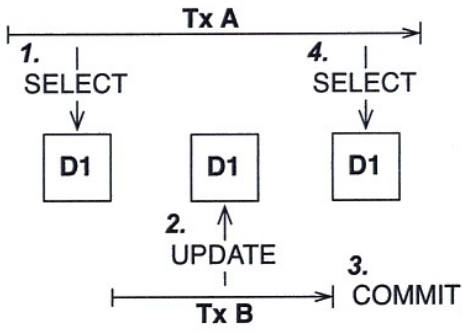
Lectura Sucia



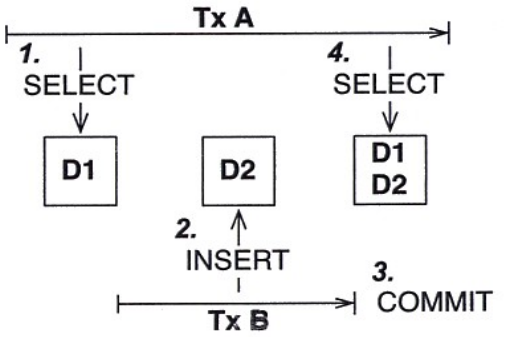
Transacciones

- **Problemas (Continuación):**
  - **Lectura No Repetible (*Non-repeatable Read*)**
    - Una transacción lee un dato dos veces y, sin que lo haya cambiado, el dato tiene dos valores distintos.
  - **Lecturas Fantasma (*Phantom Read*)**
    - Una transacción realiza una consulta dos veces y la consulta devuelve dos resultados distintos.

Lectura No Repetible



Lectura Fantasma



Niveles de Aislamiento

- El **nivel de aislamiento** permite especificar que errores se admiten y cuales no.
- Cuanto mayor sea el nivel de aislamiento mayor será el número de bloqueos.
- Deben ser soportados por el SGBD.
- Las siguientes **contantes** de `Connection` permiten definir el nivel de aislamiento.
  - `TRANSACTION_NONE` no se puede utilizar con transacciones.
  - Java no puede prevenir las actualizaciones perdidas.

Tipo (Nivel de Aislamiento)	L.S.	L.N.R.	L.F.
TRANSACTION_NONE	-	-	-
TRANSACTION_READ_UNCOMMITED	Sí	Sí	Sí
TRANSACTION_READ_COMMITED	No	Sí	Sí
TRANSACTION_REPEATABLE_READ	No	No	Sí
TRANSACTION_SERIALIZABLE	No	No	No

*"Sí" indica que, con este nivel, de aislamiento pueden ocurrir y "No" que no pueden ocurrir.

Ejemplo

```
private static void transaction() throws Exception {
   // 1. Conexión a la base de datos
   try (Connection connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD)) {

      // 2. Activación del modo transacción
      connection.setAutoCommit(false);
      connection.setTransactionIsolation(Connection.TRANSACTION_SERIALIZABLE);

      // 3. Creación de la consulta
      try (PreparedStatement statement = connection
            .prepareStatement("INSERT INTO Ejemplo (id, nombre) VALUES (0, ?)")) {
         // 4. Asignación de los valores y ejecución de la consulta
         for (String nombre : new String[] { "María", "Juan", "Luisa" }) {
            statement.setString(1, nombre);

            if (statement.executeUpdate() != 1) {
               throw new SQLException("Error inserting value");
            }

            // 5. Fuerza que se haga un rollback cuando se intente insertar
            // segundo nombre, de modo que se cancelará también la primera
            // inserción
            if (nombre.equals("Juan"))
               throw new RuntimeException();
         }

         // 5a. Commit
         connection.commit();
      } catch (Exception e) {
         // 5b. Rollback
         connection.rollback();
      }
   }
}
```

Índice

1. Introducción
2. Arquitectura
3. Consultas SQL
    i. Carga de Drivers
    ii. Conexión
    iii. Consulta
    iv. Resultados
4. Tipos de Datos
5. Transacciones
6. **Referencias**

Referencias

- **Trail: JDBC(TM) Database Access**
  - http://docs.oracle.com/javase/tutorial/jdbc/
  - **Lessons: JDBC Introduction**
    - http://docs.oracle.com/javase/tutorial/jdbc/overview/index.html
  - **Lessons: JDBC Basics**
    - http://docs.oracle.com/javase/tutorial/jdbc/basics/index.html